

Emma Doyle



She/Her

Owner @ Tiger Lily Plants

Tiger Lily Plants LLC.

Atlanta Metropolitan Area

Contact info

Tiger Lily Plants LLC.

214 connections

[LinkedIn profile](#)

<https://www.linkedin.com/in/emmeowzing/?isSelfProfile=true>

214 connections

Contact info

Your profile

[linkedin.com/in/emmeowzing](https://www.linkedin.com/in/emmeowzing)

Website

linktr.ee

(Other)

Email

emma.ann.doyle@gmail.com

Birthday

November 25

Edit contact info

[linkedin.com/in/emmeowzing](https://www.linkedin.com/in/emmeowzing)

linktr.ee

About

Owner @ Tiger Lily Plants 🌱

Featured

Post

I built a plugin for Helm charts that uses graph-based structural filtering and Python's Hypothesis testing framework to explore input configuration spaces (values.yaml) efficiently, to find the bugs you never thought to test for:

<https://lnkd.in/ektqstTY>

Most Helm testing validates static configurations and doesn't exercise the full space of configurable paths or combinations of values.

That leaves many classes of bugs among structural complexities, unaddressed before release:

- One value change can have downstream effects across multiple rendered resources.
- Individually valid settings can fail when combined.
- configurable fields exist only in templates and aren't documented in values.yaml.
- values paths are effectively dead and never change rendered output.
- Different values within the same type constraints can still produce structurally different behavior.

And all of these cases could pass a static 'helm lint'.

hypothesis-helm attempts to expand automated testing toward the full configuration surface of a chart while filtering redundant cases prior to execution.

The pipeline is roughly:

coalesce values -> pre-filter -> sort -> post-filter -> test -> validate

The tested and rendered manifests that pass hypothesis testing can then be passed through kubesecc or any other tool you already have.

I've tried my hand at benchmarking the approach against synthetic charts with varying structures and error distributions, which you can see in the [studies/-directory](#).

I've also tested it against public data, including Bitnami. See the report the tool generates for yourself:

<https://lnkd.in/eqZRNAqJ>

This example includes a lot of diagrams that give us a sense of input value sensitivity in the rendered space, as well as charts' structures.

GitHub - astrivant/hypothesis-helm: Hypothesis testing plugin for Helm
github.com

2, number of reactions

2

2 comments

astrivant/ hypothesis-helm

Hypothesis testing plugin for Helm

1 Contributor
0 Issues
0 Stars
0 Forks



astrivant/ hypothesis-helm

Hypothesis testing for Helm charts

2 Contributors
4 Used by
0 Stars
0 Forks



astrivant/ hypothesis-helm

Hypothesis testing for Helm charts

2 Contributors
4 Used by
0 Stars
0 Forks



Post I built a plugin for Helm charts that uses graph-based structural filtering and Python's Hypothesis testing framework to explore input configuration spaces (values.yaml) efficiently, to find the bugs you never thought to test for: <https://lnkd.in/ektqstTY> Most Helm testing validates static configurations and doesn't exercise the full space of configurable paths or combinations of values. That leaves many classes of bugs among structural complexities, unaddressed before release: - One value change can have downstream effects across multiple rendered resources. - Individually valid settings can fail when combined. - configurable fields exist only in templates and aren't documented in values.yaml. - values paths are effectively dead and never change rendered output. - Different values within the same type constraints can still produce structurally different behavior. And all of these cases could pass a static 'helm lint'. hypothesis-helm attempts to expand automated testing toward the full configuration surface of a chart while filtering redundant cases prior to execution. The pipeline is roughly: coalesce values -> pre-filter -> sort -> post-filter -> test -> validate The tested and rendered manifests that pass hypothesis testing can then be passed through kubesecc or any other tool you already have. I've tried my hand at benchmarking the approach against synthetic charts with varying structures and error distributions, which you can see in the studies/-directory. I've also tested it against public data, including Bitnami. See the report the tool generates for yourself: <https://lnkd.in/eqZRNAqJ> This example includes a lot of diagrams that give us a sense of input value sensitivity in the rendered space, as well as charts' structures. GitHub - astrivant/hypothesis-helm: Hypothesis testing plugin for Helm github.com 2, number of reactions 2 2 comments

GitHub - astrivant/hypothesis-helm: Hypothesis testing for Helm charts

[hypothesis-helm/docs/reports/bitnami.pdf](https://github.com/astrivant/hypothesis-helm/docs/reports/bitnami.pdf) at main · astrivant/hypothesis-helm

Post

What if a microservice could change how it works, not just how many replicas it runs?

I've been working on Polyad, a Kubernetes operator and Python SDK exploring a different approach to fault-tolerant microservices: applications that participate in their own adaptation as demand, dependencies, and available resources change.

The idea is to give developers a way to write microservices and pipelines by defining both the capabilities and its permitted responses to pressure. I'm hoping this opens the door to strategies of running production near or at-scale, while remaining adaptive and responsive to sudden impulses of traffic.

For example, a service might use high-throughput batch workers when resources are available, switch to a lower-memory implementation when constrained, or share compatible work with an authorized peer. Those transitions need to account for readiness, backpressure, and draining work already accepted in a safe manner.

Polyad coordinates this at two levels:

Within a service: SDK strategies support changes to concurrency, worker profiles, admission, and peer connections.

Across the application: Graphs and nested "PolyGraphs" describe workloads and their relationships, allowing coordinated changes to topology, routing, and capacity within administrator-defined constraints.

This complements infrastructure scaling. KEDA can add replicas; optional VPA integration lets application code understand configured resource bounds and observe and react to current Linux cgroup allocations.

The application can then adjust how it uses that capacity.

There's a graph-theory component, too: Cheeger bounds help constrain structural bottlenecks. But a well-connected graph is not an SLA guarantee. We also need to measure whether work completes on time, delivers the required capability, and meets the agreed quality level. A service that remains reachable while producing unacceptable results hasn't necessarily remained useful.

I'm exploring these interactions through local, process-based studies before spinning up cloud-based validation: comparing fixed and adaptive systems under controlled load, resource changes, and disruptions, with plots of latency, backlog, service-level attainment (SLA) that's computable at runtime, and adaptation behavior.

The intent is to make adaptation an explicit, testable part of the application contract and engineering process, not just a cascade of infrastructure reactions and PagerDuty pings after something fails.

Code and architecture (WIP):

<https://lnkd.in/edjues52>

Python SDK:

<https://lnkd.in/ep4n-NpJ>

Worker adaptation study, with graphs:

<https://lnkd.in/enpeacGx>

Service composition adaptation study, with graphs:

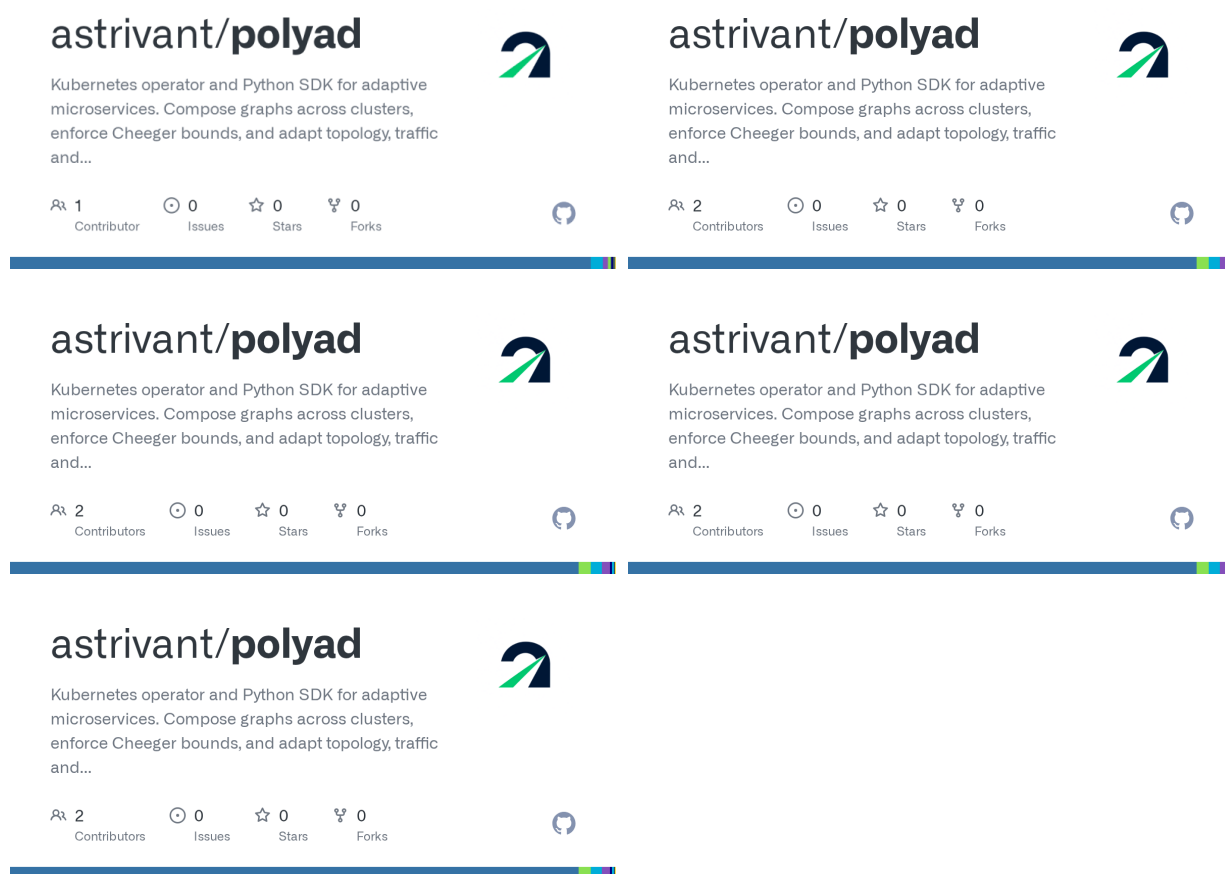
<https://lnkd.in/ew2XgdAX>

GitHub - astrivant/polyad: Kubernetes operator and Python SDK for adaptive microservices. Compose graphs across clusters, enforce Cheeger bounds, and adapt topology, traffic and capacity to application demand.

github.com

11, number of reactions

11



Post What if a microservice could change how it works, not just how many replicas it runs? I've been working on Polyad, a Kubernetes operator and Python SDK exploring a different approach to fault-tolerant microservices: applications that participate in their own adaptation as demand, dependencies, and available resources change. The idea is to give developers a way to write microservices and pipelines by defining both the capabilities

and its permitted responses to pressure. I'm hoping this opens the door to strategies of running production near or at-scale, while remaining adaptive and responsive to sudden impulses of traffic. For example, a service might use high-throughput batch workers when resources are available, switch to a lower-memory implementation when constrained, or share compatible work with an authorized peer. Those transitions need to account for readiness, backpressure, and draining work already accepted in a safe manner. Polyad coordinates this at two levels: Within a service: SDK strategies support changes to concurrency, worker profiles, admission, and peer connections. Across the application: Graphs and nested "PolyGraphs" describe workloads and their relationships, allowing coordinated changes to topology, routing, and capacity within administrator-defined constraints. This complements infrastructure scaling. KEDA can add replicas; optional VPA integration lets application code understand configured resource bounds and observe and react to current Linux cgroup allocations. The application can then adjust how it uses that capacity. There's a graph-theory component, too: Cheeger bounds help constrain structural bottlenecks. But a well-connected graph is not an SLA guarantee. We also need to measure whether work completes on time, delivers the required capability, and meets the agreed quality level. A service that remains reachable while producing unacceptable results hasn't necessarily remained useful. I'm exploring these interactions through local, process-based studies before spinning up cloud-based validation: comparing fixed and adaptive systems under controlled load, resource changes, and disruptions, with plots of latency, backlog, service-level attainment (SLA) that's computable at runtime, and adaptation behavior. The intent is to make adaptation an explicit, testable part of the application contract and engineering process, not just a cascade of infrastructure reactions and PagerDuty pings after something fails. Code and architecture (WIP): <https://lnkd.in/edjues52> Python SDK: <https://lnkd.in/ep4n-NpJ> Worker adaptation study, with graphs: <https://lnkd.in/enpeacGx> Service composition adaptation study, with graphs: <https://lnkd.in/ew2XgdAX> GitHub - astrivant/polyad: Kubernetes operator and Python SDK for adaptive microservices. Compose graphs across clusters, enforce Cheeger bounds, and adapt topology, traffic and capacity to application demand. github.com 11, number of reactions 11

GitHub - astrivant/polyad: Kubernetes operator and Python SDK for adaptive microservices. Compose graphs across clusters, enforce Cheeger bounds, and adapt topology, traffic and capacity to application demand.

[polyad/pkg/polyad-sdk at main · astrivant/polyad](#)

[polyad/studies/soul at main · astrivant/polyad](#)

[polyad/studies/nature at main · astrivant/polyad](#)

Experience

Small Business Owner

Tiger Lily Plants LLC. · Self-employed

Nov 2025 - Present · 1 yr

Atlanta, Georgia, United States · On-site

I sell orchids, begonias and carnivorous plants from both my personal collection and wholesalers to exotic plant enthusiasts.

Tiger Lily Plants LLC. | Website

Primary website for Tiger Lily Plants LLC.

TigerLilyPlants - Etsy

The Etsy store where I manage my listings.

Tiger Lily

The GitHub organization where I manage the store's blog and primary website as well as code for terrariums.



<https://www.linkedin.com/company/111622183/>

Lead Platform Engineer

SBE Vision, Inc. · Full-time

Sep 2024 - Sep 2026 · 2 yrs 1 mo

Wakefield, Massachusetts, United States · Remote

- Led platform engineering initiatives in support of SBE Vision's infrastructure across cloud and on-premise environments (prior to- and post-cloud migration).

- Migrated company to cloud infrastructure in GCP / GKE.

- Built and supported ephemeral developer environments to curtail cloud cost.

- Introduced a CI job that traversed deeply-nested Helm dependency trees in a monorepo for MR-based builds based on code change.

- Delivered a fault-tolerant and load tested CI cluster upgrade to improve reliability, scalability, and developer productivity.
 - Transitioned secrets management to ESO.
 - Delivered a Helm chart / rendered Kubernetes manifest -> Docker compose compiler to reduce time-to-delivery on PoC environments at customer sites, ensuring artifact support / generation and time cost / burden on the team stayed to a minimum.
 - Led and mentored engineers by overseeing project execution, providing technical reviews, coordinating platform work, guiding implementation decisions, and supporting team skill set growth.
 - Strengthened platform reliability / QoS, automation, and DevOps practices across infrastructure, CI/CD, and developer experience.
- GitHub - astrivant/composer: Lossily compile rendered Helm manifests to Docker compose files



<https://www.linkedin.com/company/75395048/>

GitHub - astrivant/composer: Lossily compile rendered Helm manifests to Docker compose files

Site Reliability Engineer (Security)

Klaviyo · Full-time

Feb 2024 - Sep 2024 · 8 mos

Boston, Massachusetts, United States · Hybrid

LinkedIn helped me get this job

helped me get this job

Technologies:

- Python, Go
- Puppet
- Hashicorp products, such as Terraform, Vault
- AWS (IAM, EC2)



<https://www.linkedin.com/company/2908815/>

Founder

PremiScale · Self-employed

Dec 2022 - Sep 2024 · 1 yr 10 mos

Boston, Massachusetts, United States · Hybrid

PremiScale supports projects that scale, accelerate or optimize development and deployment on private or hybrid cloud infrastructure.

Note: this project was on-hold for 2 years as I focused on personal growth in my broader career and starting my small plant business.

GitHub - premiscale/premiscale: The PremiScale autoscaler controller for Kubernetes

The PremiScale autoscaling controller for Kubernetes

GitHub - premiscale/pass-operator: A kubernetes operator that syncs and decrypts secrets from pass git repositories

A Kubernetes secrets operator based on the kopf framework for Python.



<https://www.linkedin.com/company/91593723/>

DevOps Engineer

SBE Vision, Inc. · Full-time

Aug 2022 - Feb 2024 · 1 yr 7 mos

Wakefield, Massachusetts, United States · Hybrid

SBE Vision develops an ontology based hub-and-spoke integration platform for interoperability between digital engineering tools.

Responsibilities

- Assisting customers and troubleshooting while deploying SBE Vision's microservice architecture solution to on-premise or cloud environments
- Writing internal technical documentation outlining processes, procedures and standards
- Assisted with disaster recovery in the event of on-premise power outages and hardware or software faults
- Deployed and maintained operations services
- Introduce new security, solutions and customs while practicing DevOps

Projects

- Introduced, maintained and automated an on-premise datacenter based on Rancher / RKE1 to reduce cloud costs and unify cluster access
- Introduced a scalable Kubernetes-based GitLab runners solution
- Centralized and templated Helm charts in a monorepo with supporting GitLab CI trigger framework
- Introduced an IaC solution in Terraform and GitLab CI for managing infrastructure in all public clouds

Technologies

- Helm chart authoring
- Kubernetes (on-premise Rancher RKE1, GKE / EKS / {E,G,A}CR)
- GitLab CI / static runner / kubernetes runner deployment and administration
- Ansible
- Linux KVM virtualization / automation
- Terraform
- Sonatype Nexus
- Git hook authoring
- Rancher deployment and administration
- ZFS
- TLS certificates

pre-commit hook that lints Gitlab CI configurations



<https://www.linkedin.com/company/75395048/>

pre-commit hook that lints Gitlab CI configurations

DevOps Engineer

HqO · Full-time

Aug 2021 - Jul 2022 · 1 yr

Boston, Massachusetts, United States · Hybrid

HqO creates workplace experience technology that transforms how people connect with each other and the places they work.

Responsibilities

- Writing technical documentation for engineers outlining procedures, solutions to problems, and onboarding.

Technologies

- CircleCI / Orb authoring
- GitHub Actions
- Argo CD
- Kubernetes / EKS
- Helm chart authoring / Helm chart-testing (ct)

- Spot.io
 - Infracost
 - Loft.sh / DevSpace
 - Terraform / Vault / Hashicorp Cloud
 - Artifactory
 - Git pre-commit hooks authoring
- CLI helper to filter inputs by .gitignore-like conf files in CI
- CircleCI config validate pre-commit hook for use with dynamic-continuation
- CircleCI orb pack and validate pre-commit hook



<https://www.linkedin.com/company/11386524/>

CLI helper to filter inputs by .gitignore-like conf files in CI

CircleCI config validate pre-commit hook for use with dynamic-continuation

CircleCI orb pack and validate pre-commit hook

Threat Stack, Inc

Full-time · 2 yrs

Boston, Massachusetts, United States

Tooling Solutions Engineer

Apr 2021 - Jul 2021 · 4 mos

Remote

Threat Stack is the leader in cloud security & compliance for infrastructure and applications, helping companies securely leverage the business benefits of the cloud with proactive risk identification and real-time threat detection across cloud workloads.

Projects

- Wrote a Python package and containerized Flask API to assist and support internal teams in platform rule modifications and versioning
- Continued to maintain log and diagnostics collection scripts in Bash and PowerShell for troubleshooting Linux- and Windows-based agent installations

A CLI tool for Threat Stack ruleset manipulation via API

Support Engineer

Aug 2019 - Apr 2021 · 1 yr 9 mos

On-site

Responsibilities

- Debug and reproduce reported issues with agent deployments on Kubernetes (GKE, EKS, ECS, & ECR), Docker, Windows Server and all major Linux distributions in GCP and AWS
- Document procedures and projects for other team members to reference in Confluence and GitLab
- Work cross-departmentally with Sales, Engineering, and Product to maintain customer relations and relay customer feedback
- Escalate software defects in Salesforce and Jira to product development and engineering team

Projects

- Maintainer of a log collecting package written in Bash for the Linux Agent and Support Engineering teams
- Built and maintained infrastructure monitoring projects with Python and Terraform in AWS EC2 and Lambda



<https://www.linkedin.com/company/3068295/>

A CLI tool for Threat Stack ruleset manipulation via API

Datto, Inc.

Full-time · 1 yr 2 mos

Rochester, New York Metropolitan Area · On-site

Technical Support Specialist, Blue Tier

May 2019 - Aug 2019 · 4 mos

Responsibilities

- Build strong relationships to a dedicated set of Datto's highest revenue-generating partners and MSPs
- Virtualize on-prem infrastructure in cloud-based disaster recovery tests

Technical Support Specialist, Tier 1

Jul 2018 - May 2019 · 11 mos

Projects

- Wrote and maintained a support script for use on field appliances that produced a timeline of volumes' utilization across ZFS snapshots in backups.

Responsibilities

- Administer the virtualization of Windows Server and major Linux distributions in KVM
- Troubleshoot the backup process of all major operating systems in MS Hyper-V and VMware
- Samba / SMB, NFS and iSCSI configuration in Windows Server and Linux
- Remotely assist in the maintenance of ZFS on Linux (ZoL)-based storage appliances at customer sites
- Assist in the maintenance of thousands of private ZoL-based secondary-replication backup cloud servers
- Networking fundamentals, including DNS, DHCP, VLANs, and firewall configuration

A fast (experimental) RX algorithm implementation in Python 3.9+

Parse backed-up volumes' utilization across ZFS snapshots



<https://www.linkedin.com/company/1477873/>

[A fast \(experimental\) RX algorithm implementation in Python 3.9+](#)

[Parse backed-up volumes' utilization across ZFS snapshots](#)

Evaluation Technician

Baxter International Inc. · Full-time

Aug 2017 - Jul 2018 · 1 yr

Medina, New York, United States · On-site

- Evaluate and schedule appropriate repair and rework plans on Spectrum V6 and V8 IV pumps in the Service Department
- Ensure documentation and operating procedures comply with FDA regulations
- Escalate appliances of suspected adulteration to QA Engineers
- Assist in maintaining the Service center database as a superuser



<https://www.linkedin.com/company/2371/>

Line Lead

Saint-Gobain Adfors · Full-time

Jan 2017 - Aug 2017 · 8 mos

Albion, New York, United States · On-site

- Led the production fiberglass converting and reduction process
- Set production records by optimizing machine downtime and workflow

- Used basic mathematical abilities to calculate materials, hourly rates and other relevant details
- Made sure that products were produced on time and of good quality



<https://www.linkedin.com/company/2282551/>

Undergraduate Researcher

Rochester Institute of Technology · Part-time

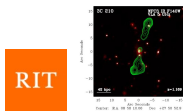
Sep 2013 - Dec 2014 · 1 yr 4 mos

Rochester, New York, United States · Remote

- Created journal quality overlays of HST and VLA FITS data with Python and Interactive Data Language (IDL)
- Cooperated with others in team-based environment to identify correlations between wavelengths from data analysis

Radio Galaxy 3C 210

An example plot.



<https://www.linkedin.com/school/165762/>

Education

Monroe Community College

Associate's Degree, Mathematics

2015 – 2016



<https://www.linkedin.com/school/19573/>

Rochester Institute of Technology

N/A, Computational Mathematics

2013 – 2014



<https://www.linkedin.com/school/165762/>

Projects

Polyad

Sep 2026 – Present

Python (Programming Language) and Graph Theory

Associated with Tiger Lily Plants LLC.



GitHub - astrivant/polyad: Kubernetes operator and Python SDK for adaptive microservices. Compose graphs across clusters, enforce Cheeger bounds, and adapt topology, traffic and capacity to application demand.

hypothesis-helm

Sep 2026 – Present

A Helm plugin that hypothesis tests your Helm chart based on types.

Helm Charts, Python (Programming Language) and +1 skill
Associated with Tiger Lily Plants LLC.



[GitHub - astrivant/hypothesis-helm: Hypothesis testing for Helm charts](#)

composer

Jul 2026 – Present

Associated with SBE Vision, Inc.



orchidarium

Sep 2025 – Present

Python (Programming Language), Docker Products and +1 skill

Associated with Tiger Lily Plants LLC.



pass-operator

Jul 2024 – Present

Python (Programming Language), Bash and +2 skills

Associated with PremiScale



Skills



Languages

Languages

English

Organizations

Organizations

Atlanta Botanical Gardens

Member · Jul 2025 – Present

Atlanta Orchid Society

Member · Jul 2025 – Present

Causes

Civil Rights and Social Action • Economic Empowerment • Human Rights • Health • Education • Animal Welfare • Poverty Alleviation • Science and Technology • Environment • Disaster and Humanitarian Relief